Cross-Chain DevSecOps: Adapting Secure Development Pipelines for Ethereum, Solana, Polkadot, and Cosmos

Ashiful Alam Shuvo	Supervised by: Safat Siddiqui
ashiful.shuvo.261@northsouth.edu	safat.siddiqui@northsouth.edu

Department of Electrical and Computer Engineering
North South University, Dhaka, Bangladesh

Abstract

Blockchain security research and tooling have developed almost entirely around Ethereum, leaving developers on Solana, Polkadot, and Cosmos without structured guidance. This paper closes that gap by proposing and empirically evaluating a cross-chain DevSecOps framework that adapts established CI/CD security pipeline practices to all four major blockchain ecosystems. The framework is evaluated using annotated smart contract datasets from each chain: 5,219 Ethereum contracts from the ScrawlD dataset and manually curated datasets for Solana (847 programs), Polkadot (312 ink! contracts), and Cosmos (428 CosmWasm contracts). Ethereum's automated pipeline achieves an overall vulnerability detection rate of 91.2%, compared to 81.2% for Solana, 77.2% for Polkadot, and 76.7% for Cosmos. The gap traces directly to the absence of mature static analysis and formal verification tools on non-Ethereum chains. The paper provides chain-specific tool configurations for every pipeline phase, a structured tooling gap analysis, and a research agenda for closing the disparity.

Keywords: DevSecOps, Blockchain Security, Smart Contracts, Cross-Chain, Solana, Polkadot, Cosmos, Ethereum, Vulnerability Detection, CI/CD Pipeline

1. Introduction

Blockchain development has moved well beyond Ethereum. Solana now processes millions of transactions daily under a fundamentally different execution model. Polkadot runs smart contracts as WebAssembly modules inside a Substrate parachain runtime. Cosmos links

hundreds of independent chains through its Inter-Blockchain Communication protocol, each running CosmWasm contracts under its own governance rules. These are not minor variations on Ethereum; they are architecturally distinct platforms with their own threat models, vulnerability classes, and security tooling needs. The security research community has been slow to catch up with this shift.

The financial consequences of that lag are well documented. The Wormhole bridge exploit in February 2022 drained 320 million dollars by exploiting a signature verification flaw in Solana's program logic. The Cashio stablecoin attack the following month extracted 52 million dollars through an account validation failure that is specific to Solana's account model and would not be caught by any Ethereum-focused security tool. The Nomad bridge incident in August 2022 cost 190 million dollars due to a flawed initialization routine in a cross-chain messaging contract. None of these attacks were novel in concept; all of them exploited patterns that, on Ethereum, would have been flagged by automated tools during development. On non-Ethereum chains, those tools simply do not exist yet.

DevSecOps offers a structured way to address this. By integrating security checks directly into CI/CD pipelines, it ensures that vulnerabilities are caught during development rather than after deployment. Dinh et al. (2025) showed that a properly configured Ethereum DevSecOps pipeline can detect around 92% of known smart contract vulnerabilities automatically. That work, however, was built entirely around Ethereum. Adapting it to Solana or Cosmos is not a matter of swapping out a few tools; the compilation process, testing infrastructure, deployment mechanism, and monitoring approach all differ significantly.

This paper builds a cross-chain DevSecOps framework that covers all four ecosystems. The specific contributions are:

- A systematic cross-chain analysis of smart contract vulnerability classes across Ethereum, Solana, Polkadot, and Cosmos, distinguishing universal vulnerabilities from platform-specific ones.
- A unified DevSecOps pipeline architecture with chain-specific tool configurations covering all phases from development through production monitoring.
- An empirical evaluation of vulnerability detection tooling across all four platforms with quantitative detection rates and a structured tooling gap analysis.
- A research agenda with practical recommendations for closing the security maturity gap between Ethereum and alternative chains.

Section 2 surveys the relevant literature. Section 3 provides background on the four ecosystems. Section 4 analyzes vulnerability classes. Section 5 presents the framework. Section 6 covers the experimental evaluation. Section 7 discusses the findings. Section 8 concludes.

2. Related Work

2.1 DevSecOps in Blockchain Environments

Dinh et al. (2025) produced the most thorough treatment of blockchain DevSecOps published so far, integrating Slither, Mythril, and ContractFuzzer into a GitLab CI/CD pipeline for Ethereum smart contracts. Their ablation study reported 92.6% vulnerability detection on the ScrawlD dataset, a result this paper uses as the Ethereum performance baseline. Their framework is, however, built entirely around Ethereum and makes no claims about other chains. Li et al. (2020) studied DevOps in consortium blockchain settings, though their focus was on operational complexity rather than security tooling. Yussupov et al. (2021) produced a systematic literature review that classified 39 DevSecOps practices and mapped them to CI/CD stages, giving a useful taxonomy without blockchain-specific tool recommendations. Santos et al. (2022) examined how DevSecOps practices relate to regulatory compliance in blockchain deployments and contributed useful compliance auditing tools, though again the underlying assumptions are EVM-centric.

2.2 Smart Contract Vulnerability Detection

Ethereum's security analysis tooling is genuinely mature. Symbolic execution tools including Oyente (Luu et al., 2016), Osiris (Torres et al., 2018), and Mythril (Mueller, 2018) established the foundational automated detection approaches for EVM contracts. Slither (Feist et al., 2019) built on those foundations with a high-performance static analysis layer covering over 90 distinct vulnerability detectors. Echidna and the Foundry fuzzer brought property-based and coverage-guided fuzzing into mainstream Solidity development workflows. Formal verification tools including KEVM (Hildenbrandt et al., 2018) and Certora's Prover now offer mathematically rigorous correctness guarantees for teams that need them.

Non-EVM chains are at a much earlier stage. Cui et al. (2022) carried out some of the first systematic security research on Solana, identifying account model vulnerabilities including missing ownership checks and unsafe cross-program invocations that have no Ethereum equivalent. The Soteria framework (Chen et al., 2022) was the first automated vulnerability scanner built specifically for Solana programs, though its coverage remains narrower than what

Slither achieves on Ethereum. For Polkadot and Cosmos, dedicated security analysis tools are essentially absent at the time of writing. Developers on those chains rely on general-purpose Rust linters and manual code review, neither of which was designed with smart contract threat models in mind. No prior publication has proposed a unified DevSecOps framework covering all four platforms within a single architecture.

3. Background: The Four Blockchain Ecosystems

3.1 Ethereum

Ethereum is the reference point for smart contract security research. Contracts are written primarily in Solidity, compiled to EVM bytecode, and executed in an account-based model where every entity on-chain, including contracts, has associated code, persistent storage, and a balance. This design enables powerful composability between contracts but also creates well-documented security hazards. Reentrancy arises because external calls transfer execution control before the calling contract updates its own state. Integer overflow was silently possible before Solidity 0.8.0 introduced built-in checked arithmetic. Delegatecall enables proxy upgrade patterns but creates storage collision vulnerabilities when implementation and proxy storage layouts are not carefully coordinated. The breadth and maturity of Ethereum's security tooling is a direct result of developers encountering all of these problems repeatedly in production over nearly a decade.

3.2 Solana

Solana's design priority is throughput. Its Proof of History consensus mechanism and the Sealevel parallel processing runtime achieve theoretical speeds of 65,000 transactions per second, making it architecturally different from Ethereum in ways that go beyond raw performance numbers. Smart contracts on Solana are called programs and are written in Rust, compiled to BPF bytecode, and executed in the Solana Virtual Machine. The most consequential difference from Ethereum is that Solana programs are stateless. They do not hold storage of their own; instead, they receive account objects as arguments at call time and operate on those accounts' data. This means a program must validate every account it receives: correct owner, correct data discriminator, required signer authorization. Missing even one of these checks opens an attack path with no real Ethereum equivalent. The Anchor framework automates much of this validation through macros, but developers who bypass Anchor's conventions or write native programs from scratch carry the full validation burden manually.

3.3 Polkadot

Polkadot connects application-specific blockchains called parachains to a central Relay Chain that provides shared security and cross-chain message passing. Smart contracts are written in ink!, a domain-specific language embedded in Rust that compiles to WebAssembly and runs inside the pallet-contracts module on Substrate-based chains. Rust's ownership and borrowing system rules out entire categories of memory vulnerabilities, which is a genuine advantage. The main ink!-specific risk comes during contract upgrades. Unlike Ethereum's proxy pattern where storage lives at the proxy address and survives implementation swaps unchanged, upgrading an ink! contract means the existing storage bytes get reinterpreted according to the new field layout. Reordering or adding fields without a migration function will cause the contract to read nonsense data silently, with no runtime error, potentially corrupting state or freezing funds.

3.4 Cosmos

Cosmos is less a single blockchain than an ecosystem of independent, interoperable chains that communicate through the Inter-Blockchain Communication protocol. Smart contracts run as CosmWasm modules, written in Rust and compiled to WebAssembly. Each Cosmos chain sets its own parameters and governance rules, which means security standards vary considerably across the ecosystem. CosmWasm's actor model processes cross-contract interactions through message queues rather than synchronous calls, which significantly reduces classical reentrancy risk. The IBC protocol introduces its own attack surface: every packet a contract sends must be handled in three possible outcomes — successful delivery, failed delivery, and timeout. A contract that does not implement all three handlers correctly can leave funds permanently locked in an escrow account with no recovery path. Governance attacks are another concern specific to Cosmos chains with on-chain voting; if a malicious governance proposal passes due to low voter turnout, it can upgrade a contract to a backdoored implementation or modify fee parameters to drain a treasury.

4. Cross-Chain Vulnerability Analysis

4.1 Vulnerabilities Common to All Four Platforms

4.1.1 Access Control Failures

Access control failures are the most prevalent and financially damaging vulnerability class across all four platforms. On Ethereum, the failure is typically a missing onlyOwner modifier. On Solana, it is a missing signer check. On Polkadot and Cosmos, it is insufficient validation of the message origin. The structural consistency of this class across architecturally distinct platforms argues for making access control verification the first and highest-priority check in any cross-chain

security pipeline. The Rari Capital exploit in 2022 (80 million dollars) and the Solend governance attack demonstrate the real-world stakes.

4.1.2 Arithmetic Errors

Arithmetic overflow and underflow appear across all four platforms. Solidity 0.8.0 added built-in protection for Ethereum contracts. Rust, used by the other three chains, panics on overflow in debug builds but wraps silently in release builds compiled with the --release flag unless developers explicitly use checked_add() and similar safe methods. Precision loss in fixed-point division and token decimal handling are further shared concerns.

4.1.3 Logic Errors

Logic errors are the hardest category to catch automatically because detection requires knowing what the contract was supposed to do. The Compound Finance incident in 2021, where a governance proposal inadvertently distributed 90 million dollars in COMP tokens to incorrect addresses, exemplifies a logic flaw that no static analyzer would catch without a formal behavioral specification. These vulnerabilities are present across all four platforms in roughly equal measure and motivate formal specification and property-based testing within the pipeline.

4.2 Platform-Specific Vulnerability Classes

4.2.1 Ethereum: Reentrancy and Delegatecall

Reentrancy exploits the EVM's synchronous call semantics: when a contract sends ETH to an external address, execution transfers to that address's fallback function before the caller has updated its own state. A malicious contract can use this window to re-enter and drain funds before the balance decrements. The DAO attack of 2016 triggered Ethereum's hard fork and remains the canonical example. Delegatecall enables upgradeable proxy patterns but introduces storage collision vulnerabilities when proxy and implementation contract storage layouts diverge. Both vulnerability classes are unique to the EVM.

4.2.2 Solana: Account Confusion and CPI Attacks

Account confusion occurs when a Solana program accepts an account of an unexpected type without verifying its owner program or data discriminator. The Cashio exploit in 2022 extracted 52 million dollars this way: the protocol treated an attacker-controlled account as a legitimate USDC mint authority. Cross-Program Invocation vulnerabilities occur when a program calls another program without adequately validating returned accounts or when privilege escalation is possible through manipulated signer seeds. Anchor substantially reduces these risks

through its constraint system, but native programs and those that bypass Anchor's defaults remain exposed.

4.2.3 Polkadot: Storage Layout Corruption

ink! contract upgrades carry a storage layout risk more subtle than Ethereum's proxy pitfalls. When an ink! contract is upgraded, the existing storage is interpreted according to the new field layout. Adding, removing, or reordering storage fields without a migration function causes the contract to read incorrect data silently, potentially locking funds or corrupting permissions. Off-chain workers, a Substrate feature that runs computation outside consensus, introduce an additional attack surface through their signed transaction submissions to on-chain state.

4.2.4 Cosmos: IBC Handling and Governance Attacks

CosmWasm contracts that use IBC must handle three outcomes for every sent packet: successful acknowledgement, failed acknowledgement, and timeout. Failing to handle any outcome can permanently lock escrowed funds. Governance attacks are a structural risk in Cosmos chains: a malicious proposal can pass and upgrade a contract to a compromised implementation if validator participation is too low to block it. The Juno network governance controversy in 2022 illustrated how governance mechanisms can be weaponized in ways that no smart contract audit would prevent.

Table 1: Cross-Chain Smart Contract Vulnerability and Tooling Comparison

Criterion	Ethereum	Solana	Polkadot	Cosmos
Language	Solidity / Vyper	Rust	ink! (Rust DSL)	Rust
Runtime	EVM	SVM / BPF	Wasm / Substrate	Wasm / CosmWasm
Reentrancy Risk	High	Low	Medium	Medium
Arithmetic Safety	Pre-0.8.0 risk	Rust checked arith.	Rust safe by default	Rust safe by default
Access Control	Modifier-based	Signer verification	Origin/caller checks	Admin / governance
Static Analysis	Slither, Mythril (mature)	clippy, Soteria (emerging)	clippy, cargo-audit (limited)	cosmwasm-check (limited)
Formal Verification	KEVM, ZEUS, Certora	None available	None available	None available
Fuzzing	Echidna, Foundry (mature)	cargo-fuzz (moderate)	cargo-fuzz (basic)	cargo-fuzz (basic)
CI/CD Tooling	Hardhat, Foundry, Truffle	Anchor, Cargo	cargo-contract	beaker, wasmd
Testnet	Goerli, Sepolia	Devnet, Testnet	Rococo, Westend	Theta Testnet

5. Cross-Chain DevSecOps Framework

5.1 Architecture and Design Principles

The framework uses a two-layer architecture. The outer layer is a platform-agnostic pipeline core defining security activities, quality gate requirements, and evidence generation standards applicable to any smart contract project. The inner layer contains four chain-specific adapter configurations that translate these abstract requirements into concrete tool invocations, test commands, and monitoring integrations.

Three principles guided the design. First, security controls must be pipeline-blocking: a high-severity static analysis finding must halt the CI pipeline, not trigger an advisory notice. Second, every phase must generate auditable evidence recording what was checked, by what tool, and with what result. Third, where mature tooling is unavailable, the gap must be surfaced explicitly rather than papered over by superficial checks that create false confidence.

5.2 Phase 1: Secure Code Development

Security in the development phase starts with choosing the right tools and libraries before a single line of contract code is written. On Ethereum, Remix IDE with its integrated Solidity analyzer catches common mistakes as developers type, and OpenZeppelin Contracts should be the default source for access control patterns, token standards, and upgrade proxy implementations. There is no good reason to reimplement these from scratch when audited versions already exist. On Solana, the standard development setup is Visual Studio Code with rust-analyzer alongside Anchor's language server; Anchor's constraint macros for account validation should be used consistently and not bypassed unless there is a specific technical reason. Polkadot developers working with ink! use cargo-contract and the OpenBrush library, which provides ink! equivalents of the most commonly needed OpenZeppelin contract patterns. On Cosmos, the beaker scaffolding tool and cw-plus library fill the same role.

Before writing any contract logic, security requirements should be documented: which vulnerability classes the contract must defend against, the trust model for every external interaction, and for contracts that participate in IBC or bridge protocols, a clear list of which chains and contracts are considered trusted message sources.

5.3 Phase 2: Build and Static Analysis

Static analysis runs automatically on every commit and pull request, with high-severity findings blocking the merge. Table 2 maps the specific toolchain for each phase across all four platforms.

Table 2: Cross-Chain DevSecOps Pipeline Phase and Tool Mapping

Phase	Ethereum	Solana	Polkadot	Cosmos
Code	Remix, Hardhat, OpenZeppelin	Anchor, rust-analyzer	cargo-contract, OpenBrush	beaker, cw-plus
Build	solc, Hardhat compile	anchor build, cargo build-bpf	cargo +nightly build	cargo wasm, optimizer
SAST	Slither, Mythril, Semgrep	cargo clippy, Soteria	cargo clippy, cargo audit	cosmwasm-check, clippy
Test	Foundry, Truffle, Echidna	Anchor test, cargo-fuzz	drink!, substrate-node	cw-multi-test, beaker
Deploy	Hardhat Ignition, Truffle	anchor deploy	cargo-contract instantiate	beaker deploy, wasmd
Monitor	Etherscan, Tenderly, Defender	Helius, Solana Explorer	Subscan, Polkadot.js	Mintscan, CosmJS

On Ethereum, Slither and Mythril run in parallel. Slither's pattern-based detectors handle access control failures, arithmetic issues, and common Solidity anti-patterns efficiently and with relatively few false positives. Mythril uses symbolic execution to explore deeper execution paths in multi-contract interaction scenarios that Slither's pattern matching cannot reach. Running both together provides substantially better coverage than either tool alone. On Solana, the static analysis layer is cargo clippy combined with Soteria. Clippy enforces Rust code quality standards; Soteria adds Solana-specific checks for missing account validation that Clippy has no knowledge of. On Polkadot and Cosmos, the static analysis layer is cargo clippy and cargo audit. Cargo audit checks all dependencies against the RustSec Advisory Database, which matters because many vulnerabilities on Rust-based chains originate in upstream dependencies rather than contract code itself. These tools provide a useful baseline, but their coverage is significantly narrower than Slither's, and the framework documentation flags this explicitly so teams know that manual review carries more weight on these platforms.

5.4 Phase 3: Dynamic Testing and Fuzzing

Dynamic testing catches vulnerabilities that static analysis cannot see because static analysis does not execute the code. On Ethereum, Foundry is the recommended testing environment. Its Solidity-native test syntax avoids the overhead of JavaScript test clients, the forge

runner executes quickly, and the built-in fuzzer generates inputs that push contract code into paths that handwritten tests would never reach. Echidna complements Foundry by letting developers write invariants that must hold across all possible input sequences, providing the strongest available detection for logic errors. On Solana, the Anchor testing framework supports TypeScript and Rust test clients against a local solana-test-validator instance. An important Solana-specific practice is account configuration fuzzing: tests should deliberately construct accounts with wrong owners, wrong discriminators, and uninitialized data, then confirm the program rejects each configuration correctly. This directly tests the account validation logic responsible for the majority of Solana exploits. On Polkadot, the drink! framework simulates the pallet-contracts runtime off-chain without requiring a full Substrate node. On Cosmos, cw-multi-test provides the most complete CosmWasm testing environment available, including simulation of IBC message passing, making it the only practical way to test IBC acknowledgement and timeout handling paths before deployment.

5.5 Phase 4: Deployment Security

Testnet deployment is mandatory before any mainnet promotion in this framework. Testnet runs expose environment-specific issues that local testing misses and provide a final opportunity to verify contract behavior before real assets are at risk. Deployment scripts live in version control alongside the contract code and include post-deployment steps that compare the deployed bytecode hash against the locally compiled artifact, guarding against supply chain attacks that modify build outputs. For any contract managing significant value, multi-signature deployment approval is required. Gnosis Safe on Ethereum, Squads on Solana, and comparable tools on Polkadot and Cosmos ensure that deploying or upgrading a contract requires sign-off from multiple authorized keys, so no single compromised account can push a malicious change to production.

5.6 Phase 5: Runtime Monitoring

Ethereum has the most developed monitoring infrastructure of the four platforms. OpenZeppelin Defender Sentinel watches transactions in real time and triggers alerts based on configurable conditions such as large token transfers, ownership changes, or calls to sensitive functions. Tenderly provides detailed transaction tracing and lets teams simulate hypothetical attack transactions to understand their impact before they happen. Forta Network runs a decentralized set of detection bots that watch Ethereum transactions for known attack signatures. On Solana, Helius and QuickNode offer webhook-based transaction monitoring, and custom scripts using the Solana Web3.js WebSocket API enable low-latency alerts on account state

changes. On Polkadot, the Polkadot.js API supports programmatic monitoring of parachain events alongside Subscan's block explorer APIs. On Cosmos, CosmJS provides the equivalent monitoring layer and Mintscan offers explorer services across most major chains.

Incident response planning cannot wait until something goes wrong. Before any contract goes live, the team needs documented answers to: who has authority to trigger an emergency pause or privilege revocation, what the escalation path is if that person is unavailable, and how a security incident will be disclosed publicly.

6. Experimental Evaluation

6.1 Dataset and Setup

The Ethereum evaluation used the ScrawlD dataset (Yadav et al., 2022), a publicly available collection of 5,219 real-world contracts annotated with vulnerability labels covering reentrancy, arithmetic overflow, access control, timestamp dependency, and logic errors. This dataset has been used and validated in prior published work. Comparable annotated datasets do not exist for the other three chains, so they were constructed manually. For Solana, 847 programs were collected from the public program registry and from contracts associated with previously reported security incidents; 623 of these were annotated with vulnerability labels based on published audit reports and post-mortem analyses. For Polkadot, 312 ink! contracts were drawn from the ink! examples repository and the OpenBrush library, with 218 annotated. For Cosmos, 428 CosmWasm contracts from the CosmWasm repository and deployed chains were collected, with 307 annotated. The pipeline ran in GitLab CI/CD with separate configurations per chain. Static analysis stages were set to block on high-severity findings. Dynamic testing used Hardhat and Foundry for Ethereum, solana-test-validator for Solana, substrate-contracts-node for Polkadot, and a local wasmd instance for Cosmos.

6.2 Results

Table 3 presents the vulnerability detection rates. Table 4 summarizes the tooling maturity gap across security capabilities.

Table 3: Vulnerability Detection Rates by Platform and Vulnerability Type (%)

Vulnerability Type	Ethereum (%)	Solana (%)	Polkadot (%)	Cosmos (%)	Avg. (%)
Reentrancy	94.3	72.1	68.4	70.9	**76.4**
Arithmetic Overflow	97.8	91.2	90.5	89.7	**92.3**

Vulnerability Type	Ethereum (%)	Solana (%)	Polkadot (%)	Cosmos (%)	Avg. (%)
Access Control	89.6	83.4	80.1	78.6	**82.9**
Timestamp Dependency	91.5	N/A	77.3	74.2	**81.0**
Cross-Contract Calls	88.7	79.3	75.6	76.8	**80.1**
Logic / Token Freeze	85.2	80.1	71.4	69.8	**76.6**
Overall	**91.2**	**81.2**	**77.2**	**76.7**	**81.6**

Table 4: Security Tooling Gap Analysis (green = high maturity, orange = medium, red = low/absent)

Security Capability	Ethereum	Solana	Polkadot	Cosmos
Static Analysis Coverage	High	Medium	Low	Low
Formal Verification	Available	None	None	None
Fuzzing Integration	Mature	Moderate	Basic	Basic
CI/CD Maturity	High	Moderate	Low	Low
Vulnerability Dataset Size	Large	Small	Very Small	Very Small
Security Audit Ecosystem	Extensive	Growing	Limited	Limited

Ethereum's 91.2% overall rate reflects the combined contribution of Slither, Mythril, and Echidna. Slither handles access control and arithmetic reliably. Mythril explores multi-contract paths that pattern detection misses. Echidna's invariant testing catches logic errors that neither static tool can detect without a specification. Solana's 81.2% rate is reasonable given its tooling stage, but the 72.1% reentrancy detection rate reveals a meaningful gap: CPI-based reentrancy on Solana is less reliably detected than EVM reentrancy is by Slither. Polkadot and Cosmos achieve 77.2% and 76.7% respectively, both limited primarily by the absence of platform-specific static analyzers. Their detection rates represent approximately the ceiling of what general-purpose Rust tooling can achieve without specialized knowledge of ink! or CosmWasm vulnerability patterns. The most striking finding in Table 4 is that formal verification is available only on Ethereum. For contracts on the other three chains managing significant assets, this absence makes manual auditing and property-based testing correspondingly more critical, since they must compensate for what formal methods would otherwise guarantee. Pipeline execution times averaged 4.2 minutes for Ethereum, 6.8 minutes for Solana, 7.1 minutes for Polkadot, and 6.5 minutes for Cosmos, with the longer Rust-based times attributable to Rust's compilation overhead.

7. Discussion

7.1 What the Security Maturity Gap Means in Practice

The raw detection rate numbers tell only part of the story. A 10 to 15 percentage point gap sounds manageable until you consider what fills the space around the tools on Ethereum that does not exist elsewhere. When an Ethereum developer is unsure whether a code pattern is safe, they can search years of documented vulnerability reports, check Slither's 90-plus detector descriptions, consult the OpenZeppelin audit archive, or reach out to a genuinely large pool of EVM-trained security engineers. None of that infrastructure exists at comparable depth for Solana, Polkadot, or Cosmos. The tooling deficit compounds into a knowledge deficit and a talent deficit, all three reinforcing each other. Looking at the major incidents of 2022 makes this concrete: Wormhole ($320 million) exploited a Solana-specific signature verification pattern, Cashio ($52 million) exploited Solana account model assumptions, and Nomad ($190 million) exploited a cross-chain message initialization flaw. These were not exotic zero-days. They were straightforward mistakes of the kind that Ethereum tooling has been catching automatically for years.

7.2 Toward Platform-Agnostic Security Standards

Something worth noting from building this framework is that the actual security requirements are remarkably consistent across all four platforms. Access control, input validation, arithmetic correctness, safe upgrade management: these show up everywhere regardless of whether the contract is written in Solidity, Rust, or ink!. The tools differ, the syntax differs, the runtime semantics differ, but the things developers need to get right are largely the same. This suggests that a platform-agnostic smart contract security standard is both feasible and worth pursuing, similar in spirit to what OWASP did for web application security. Such a standard would specify what controls are required without prescribing specific tools, letting each chain's ecosystem demonstrate compliance through whatever means are appropriate. Auditors would have a shared baseline to work from. Developers moving between chains would carry transferable knowledge. Institutional investors evaluating projects across different chains would have a common reference point for assessing security posture.

7.3 The Potential Role of AI-Assisted Analysis

Given how slowly platform-specific security tooling develops, AI-assisted vulnerability detection deserves serious consideration as a near-term supplement. Early research suggests that large language models trained on smart contract code can identify vulnerability patterns across platforms, catching issues in Solana programs that share structural similarities with known

Ethereum vulnerabilities even without explicit Solana-specific training. This kind of cross-platform generalization is exactly what the tooling gap makes valuable. Adding an AI analysis step to the CI pipeline for Polkadot and Cosmos contracts would not replace the need for dedicated static analyzers, but it could raise the detection floor while the community builds those tools. Autonomous monitoring agents that correlate on-chain anomalies with known attack signatures are similarly useful at the runtime monitoring phase, where non-Ethereum chains currently have the weakest coverage.

7.4 Limitations

Several limitations of this study should be stated clearly. The datasets for Solana, Polkadot, and Cosmos were assembled manually and are much smaller than the Ethereum dataset. Manual vulnerability annotation is inherently subjective, and different annotators would likely disagree on borderline cases. The detection rates reported here capture only what automated pipeline tools identify; professional audits regularly uncover vulnerabilities that no automated tool flags, and that contribution is not reflected in these numbers. The 91.2% Ethereum figure, in particular, should not be read as a general claim about Ethereum contract security. It means the pipeline detected 91.2% of the labeled vulnerabilities in the ScrawlD sample, which is a curated set of known vulnerability patterns. Finally, tool recommendations go stale quickly in this space. What is described here reflects the state of the ecosystem at the time of writing and will need revisiting as new tools are released.

8. Conclusion

This paper set out to answer a straightforward question: can the DevSecOps practices that have made Ethereum smart contract development substantially safer be adapted to work on Solana, Polkadot, and Cosmos? The answer is yes, but with important caveats that the blockchain security community needs to take seriously.

The empirical results show a real and measurable security maturity gap. Ethereum's pipeline achieves 91.2% vulnerability detection using tools that have been developed and refined over nearly a decade. Solana reaches 81.2%, Polkadot 77.2%, and Cosmos 76.7%, not because those chains have worse code quality, but because they lack equivalent tools. Formal verification does not exist for any of the three. Static analysis is limited to general Rust linters rather than platform-aware analyzers. Vulnerability datasets are small, making it harder to train and

benchmark detection tools. Each of these gaps is a solvable engineering problem, but solving them requires deliberate investment that has not yet happened at scale.

What does transfer across all four platforms is the underlying security logic. Access control mistakes, arithmetic errors, unsafe upgrade patterns, and insufficient input validation appear in every ecosystem. A developer who understands these patterns deeply is more useful on any chain than one who has only memorized tool outputs for a single platform. Building on that observation, this paper argues for a platform-agnostic security standard that specifies what smart contract security requires without locking teams into Ethereum-specific tools.

The most valuable direction for future research is the development of dedicated static analysis and formal verification tools for Solana, Polkadot, and Cosmos. AI-assisted cross-platform vulnerability detection is a useful short-term supplement while those tools are built. In the meantime, teams deploying high-value contracts on non-Ethereum chains should treat the tooling gap as a known risk and compensate through more rigorous manual review, more extensive property-based testing, and more conservative upgrade and deployment practices than they might apply on Ethereum.

References

[1] N. Dinh et al., "Enhancing Smart Contract Security Through DevSecOps: An Adaptive Approach for Vulnerability Detection," IEEE Access, DOI: 10.1109/ACCESS.2025.3606572, 2025.

[2] L. Luu et al., "Making smart contracts smarter," in Proc. ACM SIGSAC CCS, pp. 254-269, 2016.

[3] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in Proc. IEEE/ACM WETSEB, pp. 8-15, 2019.

[4] E. Hildenbrandt et al., "KEVM: A complete formal semantics of the Ethereum virtual machine," in Proc. IEEE CSF, pp. 204-217, 2018.

[5] B. Jiang, Y. Liu, and W. K. Chan, "ContractFuzzer: Fuzzing smart contracts for vulnerability detection," in Proc. IEEE/ACM ASE, pp. 259-269, 2018.

[6] S. Kalra et al., "Zeus: Analyzing the safety of smart contracts," in Proc. NDSS, pp. 1-12, 2018.

[7] I. Nikolic et al., "Finding the greedy, prodigal, and suicidal contracts at scale," in Proc. ACSAC, pp. 653-663, 2018.

[8] P. Tsankov et al., "Securify: Practical security analysis of smart contracts," in Proc. ACM SIGSAC CCS, pp. 67-82, 2018.

[9] S. Tikhomirov et al., "SmartCheck: Static analysis of Ethereum smart contracts," in Proc. WETSEB, pp. 9-16, 2018.

[10] C. F. Torres, J. Schutte, and R. State, "Osiris: Hunting for integer bugs in Ethereum smart contracts," in Proc. ACSAC, pp. 664-676, 2018.

[11] M. Fu, J. Pasuksmit, and C. Tantithamthavorn, "AI for DevSecOps: A Landscape and Future Opportunities," arXiv:2404.04839, 2024.

[12] Y. C. S. Yadav, S. Kumar, and A. Karkare, "ScrawlD: A dataset of real world Ethereum smart contracts labeled with vulnerabilities," arXiv:2202.11409, 2022.

[13] Z. Zhou et al., "SoK: Decentralized Finance (DeFi) Incidents," in Proc. IEEE S&P, pp. 1444-1461, 2023.

[14] A. Zamyatin et al., "XCLAIM: Trustless, interoperable, cryptocurrency-backed assets," in Proc. IEEE S&P, pp. 193-210, 2019.

[15] B. Mueller, "Mythril: A Framework for Bug Hunting on the Ethereum Blockchain," GitHub Repository, 2018.

[16] A. Santos, P. Silva, and L. Silva, "Holding on to Compliance While Adopting DevSecOps: An SLR," Electronics, vol. 11, no. 22, p. 3707, 2022.

[17] S. Li, Q. Xu, and P. Hou, "Exploring the challenges of developing and operating consortium blockchains," in Proc. ACM ICPC, pp. 398-404, 2020.

[18] W. Cui et al., "On the Security of Smart Contracts on Solana," in Proc. USENIX Security, 2023.

[19] N. MacDonald and I. Head, "DevSecOps: How to seamlessly integrate security into DevOps," Gartner Rep. G00315283, 2016.

[20] G. Pillala, D. Azarpazhooh, and S. Baxter, "DevSecOps Sentinel: GenAI-Driven Agentic Workflows for Supply Chain Security," Computer and Information Science, vol. 18, no. 1, p. 39, 2025.

[21] C. Liu et al., "ReGuard: Finding reentrancy bugs in smart contracts," in Proc. ICSE-Companion, pp. 65-68, 2018.

[22] Datadog, "DevSecOps Maturity Model," Datadog, New York, 2023.

[23] DeFiLlama, "Total Value Locked Dashboard," https://defillama.com/, Accessed Jan. 2024.

[24] OpenZeppelin, "OpenZeppelin Contracts," https://www.openzeppelin.com/contracts, Accessed Feb. 2024.